

Quantile GAM Regression

GAM.jl Contributors

Table of contents

Introduction	1
Setup	1
Load heteroscedastic data	2
Fit a single quantile GAM	2
Median regression ($\tau = 0.5$)	2
Compare with mean regression	3
Fit multiple quantiles with <code>mqqam</code>	3
Quantile crossing check	4
Verify quantile coverage	4
Individual quantile model summaries	5
Comparison table	8

Introduction

This vignette demonstrates quantile GAM regression using the R `qgam` package, fitting the same heteroscedastic data as the Julia vignette for comparison.

Setup

```
library(qgam)
```

```
Loading required package: mgcv
```

```
Loading required package: nlme
```

```
This is mgcv 1.9-3. For overview type 'help("mgcv-package")'.
```

```
library(mgcv)
```

Load heteroscedastic data

$$y = \sin(2\pi x) + (0.2 + 0.8x)\varepsilon, \quad \varepsilon \sim N(0, 1)$$

```
dat <- read.csv("../data.csv")
x <- dat$x
y <- dat$y
n <- nrow(dat)
cat("Data: n =", n, "\n")
```

Data: n = 500

```
cat("y range:", range(y), "\n")
```

y range: -3.150623 2.272297

Fit a single quantile GAM

Median regression (= 0.5)

```
m_50 <- qgam(y ~ s(x, k = 20, bs = "cr"), data = dat, qu = 0.5)
```

Estimating learning rate. Each dot corresponds to a loss evaluation.
qu = 0.5.....done

```
summary(m_50)
```

Family: elf
Link function: identity

Formula:
y ~ s(x, k = 20, bs = "cr")

Parametric coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	0.005622	0.026322	0.214	0.831

Approximate significance of smooth terms:

	edf	Ref.df	Chi.sq	p-value
s(x)	7.648	9.445	729.6	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) = 0.55 Deviance explained = 48%
-REML = 475.6 Scale est. = 1 n = 500

```
yhat_50 <- predict(m_50)
cat("Median regression fitted range:", range(yhat_50), "\n")
```

Median regression fitted range: -1.081852 0.946946

Compare with mean regression

```
m_mean <- gam(y ~ s(x, k = 20, bs = "cr"), data = dat)
yhat_mean <- predict(m_mean)

cat("Mean regression fitted range:", range(yhat_mean), "\n")
```

Mean regression fitted range: -1.048656 0.9559633

```
cat("Correlation (mean vs median):", cor(yhat_mean, yhat_50), "\n")
```

Correlation (mean vs median): 0.9988322

Fit multiple quantiles with mqgam

```
quantiles <- c(0.1, 0.25, 0.5, 0.75, 0.9)

mq <- mqgam(y ~ s(x, k = 20, bs = "cr"), data = dat, qu = quantiles)
```

Estimating learning rate. Each dot corresponds to a loss evaluation.

```
qu = 0.5.....done
qu = 0.25.....done
qu = 0.75.....done
qu = 0.1.....done
qu = 0.9.....done
```

Examine fitted ranges for each quantile:

```
for (qu in quantiles) {
  yhat <- qdo(mq, qu, predict, newdata = dat)
  cat(sprintf(" = %s: fitted range [%.3f, %.3f]\n", qu, min(yhat), max(yhat)))
}
```

```
= 0.1: fitted range [-2.004, 0.412]
= 0.25: fitted range [-1.522, 0.654]
= 0.5: fitted range [-1.082, 0.947]
= 0.75: fitted range [-0.556, 1.277]
= 0.9: fitted range [-0.134, 1.575]
```

Quantile crossing check

```
ord <- order(x)
predictions <- sapply(quantiles, function(qu) {
  qdo(mq, qu, predict, newdata = dat)[ord]
})

n_crossings <- sum(apply(predictions, 1, is.unsorted))
cat(sprintf("Quantile crossings: %d out of %d observations (0.1f%%)\n",
            n_crossings, n, 100 * n_crossings / n))
```

Quantile crossings: 0 out of 500 observations (0.0%)

Verify quantile coverage

```
for (qu in quantiles) {
  yhat <- qdo(mq, qu, predict, newdata = dat)
  coverage <- mean(y < yhat)
  cat(sprintf(" = %s: empirical coverage = %.3f (target = %.2f)\n", qu, coverage, qu))
}
```

```
= 0.1: empirical coverage = 0.076 (target = 0.10)
= 0.25: empirical coverage = 0.234 (target = 0.25)
= 0.5: empirical coverage = 0.500 (target = 0.50)
= 0.75: empirical coverage = 0.758 (target = 0.75)
= 0.9: empirical coverage = 0.912 (target = 0.90)
```

Individual quantile model summaries

```
for (qu in quantiles) {
  cat(sprintf("\n--- Quantile = %s ---\n", qu))
  print(summary(qdo(mq, qu)))
}
```

```
--- Quantile = 0.1 ---
```

```
Family: elf
Link function: identity
```

```
Formula:
y ~ s(x, k = 20, bs = "cr")
```

```
Parametric coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept) -0.8010      0.0404  -19.83  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Approximate significance of smooth terms:
```

```
              edf Ref.df Chi.sq p-value
s(x) 6.361  7.888  631.2  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
R-sq.(adj) = 0.456   Deviance explained = 78.7%
-REML = 677.54   Scale est. = 1           n = 500
```

```
--- Quantile   = 0.25 ---
```

```
Family: elf
Link function: identity
```

```
Formula:
y ~ s(x, k = 20, bs = "cr")
```

```
Parametric coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept) -0.39329    0.03368  -11.68   <2e-16 ***
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
Approximate significance of smooth terms:
```

```
      edf Ref.df Chi.sq p-value
s(x) 6.954  8.598  632.5  <2e-16 ***
---
```

```
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

```
R-sq.(adj) = 0.529   Deviance explained = 57%
-REML = 547.66   Scale est. = 1           n = 500
```

```
--- Quantile   = 0.5 ---
```

```
Family: elf
Link function: identity
```

```
Formula:
y ~ s(x, k = 20, bs = "cr")
```

```
Parametric coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept) 0.005622    0.026322   0.214   0.831
---
```

```
Approximate significance of smooth terms:
```

```
      edf Ref.df Chi.sq p-value
s(x) 7.648  9.445  729.6  <2e-16 ***
---
```

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) = 0.55 Deviance explained = 48%

-REML = 475.6 Scale est. = 1 n = 500

--- Quantile = 0.75 ---

Family: elf

Link function: identity

Formula:

y ~ s(x, k = 20, bs = "cr")

Parametric coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	0.4141	0.0295	14.04	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:

	edf	Ref.df	Chi.sq	p-value
s(x)	7.889	9.706	409.2	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) = 0.522 Deviance explained = 58.5%

-REML = 516.67 Scale est. = 1 n = 500

--- Quantile = 0.9 ---

Family: elf

Link function: identity

Formula:

y ~ s(x, k = 20, bs = "cr")

Parametric coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	0.76904	0.03258	23.61	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Approximate significance of smooth terms:

```

      edf Ref.df Chi.sq p-value
s(x) 7.309  8.993  313.8  <2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

R-sq.(adj) =  0.474   Deviance explained = 79.9%
-REML = 617.69   Scale est. = 1           n = 500

```

Comparison table

Feature	R qgam	Julia GAM.jl
Single quantile	<code>qgam(formula, data=dat, qu=)</code>	<code>qgam(formula, data; qu=)</code>
Multiple quantiles	<code>mqgam(formula, data=dat, qu=c(...))</code>	<code>mqgam(formula, data; qu=[...])</code>
Access fits	<code>mq\$fit[["0.5"]]</code>	<code>mq.fits[0.5]</code>
Loss function	ELF	ELF
Smoothing	Automatic	Automatic